

《企业级应用软件设计与开发》期末大作业报告

企业项目协同系统的智能运营 Agent 改造

方向二：企业级应用软件的 Agent 改造

字段	内容
课程名称	企业级应用软件设计与开发
项目名称	企业项目协同系统的智能运营 Agent 改造
方向	方向二：企业级应用软件的 Agent 改造
学号	2025302979
姓名	程辉高
专业	计算机技术
指导教师	戚欣
提交日期	2026 年 6 月 22 日

项目摘要与交付物概览

本项目以“企业项目协同系统”为模拟企业级应用场景，先实现客户、项目、合同、工单、审批、用户权限和审计日志等传统后台能力，再用智能运营 Agent 进行增强。系统允许用户用自然语言提出业务目标，例如生成经营周报、分析高风险项目、自动派单和审批合同；Agent 将这些目标拆解为工具调用，并把每一步 observation 写入运行轨迹。

项目采用 SDD（规格驱动开发）方法组织交付。Product Spec 用于定义业务目标和验收标准，Architecture Spec 用于约束系统边界和数据流，API Spec 用于固化接口、权限和工具调用协议。实现上采用 Python 标准库 HTTP 服务、SQLite、Function Calling 工具注册表、Agent 记忆表、MCP stdio JSON-RPC 工具服务、Benchmark 脚本和 smoke test，形成从规格到代码、从演示到评估的闭环。

交付物	路径/形式	作用
Web 应用	run.py / app/server.py / app/static	提供登录、业务模块、智能助手和可观测日志界面
Agent 核心	app/agent/agent.py	执行 plan -> tool -> observation -> answer 的闭环
工具层	app/agent/tools.py	实现经营指标、风险分析、派单、审批、知识库检索等工具
规格文档	docs/specs/*.md	对应 SDD 的 Product/Architecture/API Spec
评估脚本	tests.py / scripts/benchmark_agent.py / scripts/smoke_web.py	验证功能、Agent 行为和 Web 可用性
最终报告	docs/CS599_大作业报告_程辉高.docx	对应课程要求的书面报告

一、选题背景与设计思想

1.1 问题定义

企业级应用软件的核心价值不只在于增删改查，而在于支撑组织内的协作、流程、权限和决策。传统后台系统通常能把客户、项目、合同、工单等数据记录下来，但用户仍然需要在多个页面之间来回切换，自行判断风险、自行整理报告、自行依据经验分派任务。随着业务数据不断积累，这类“人工跨表分析”和“人工流程路由”会成为效率瓶颈。

本项目选择方向二“企业级应用软件的 Agent 改造”，将一个模拟项目型企业后台升级为智能运营 Agent 系统。改造目标不是为了简单增加聊天窗口，而是把已有业务数据、规则和操作封装为受控工具，使 Agent 能在权限和审计约束下完成具体业务任务。

核心设计思想：Agent 不直接绕过业务系统操作数据库，而是通过工具注册表调用受控函数；每次调用都携带参数、产生 observation、写入 agent_steps 和 audit_logs，从而兼顾智能化、可解释性和企业级可治理性。

1.2 原始系统现状与痛点

痛点	原始系统表现	造成的问题	Agent 改造目标
数据分散	项目、合同、工单和审批数据分布在多个模块	项目经理需要人工跨页面判断风险	Agent 自动汇聚多表数据并输出风险原因
流程依赖经验	工单派单依赖项目经理记忆 人员负载	紧急工单容易延迟或分配不均	route_ticket 工具按规则选择处理人并更新状态
报告成本高	经营周报需要复制看板、风险和待办数据	管理层获取经营态势滞后	generate_weekly_report 自动生成结构化周报
知识难复用	审批、派单、风险规则散落在制度或经验中	新成员难以稳定执行规则	search_knowledge 将规则检索纳入 Agentic RAG
可解释性不足	传统日志只记录用户操作	难解释 Agent 为什么这样做	agent_runs/agent_steps 保存工具链路和 observation

1.3 项目价值

- 业务价值：将常见项目运营任务从“人工查表”提升为“目标驱动”，降低周报、风险分析和派单的人工成本。
- 工程价值：展示企业级系统中 Agent 与 RBAC、审计日志、业务 API、数据库事务和错误回退的结合方式。
- 课程价值：覆盖 SDD、Function Calling、MCP、记忆机制、Agentic RAG、多步骤推理、可观测性与评估等核心技术要素。
- 演示价值：即使远程 LLM API 不可用，系统也能通过 deterministic fallback planner 完成完整 Demo，避免答辩现场不可控。

1.4 技术路线

阶段	关键任务	产物	对应课程要求
规格阶段	梳理角色、痛点、用户故事和验收项	Product Spec	SDD 规格驱动开发
架构阶段	划分 Web/API/Agent/Tool/Memory/ Data 层	Architecture Spec	系统架构与 Agent 交互流程
接口阶段	定义鉴权、业务资源、Agent、 MCP 工具接口	API Spec	API 规格与可执行约束
实现阶段	实现企业后台、Agent 核心循环和 工具注册表	Python + SQLite 代码	关键实现与代码展示
评估阶段	编写 tests、benchmark、smoke test	测试报告和截图	测试与评估

二、Specs 规格文档

2.1 SDD 方法落地方式

SDD 的核心是先把“要做什么、为什么做、怎么验收”写成可执行规格，再让实现围绕规格闭环推进。本项目不是先写代码再补文档，而是围绕 Product Spec、Architecture Spec、API Spec 建立可追踪关系：用户故事映射到工具，工具映射到 API 和数据库，测试脚本再反向验证这些能力是否成立。

规格	主要问题	关键内容	落地位置
Product Spec	系统要解决什么问题	角色、痛点、用户故事、非功能要求、验收标准	docs/specs/product_spec.md
Architecture Spec	系统如何组织	分层架构、Agent 流程、数据流、改造前后对比	docs/specs/architecture_spec.md
API Spec	外部和内部如何交互	鉴权、业务资源、Agent API、MCP/Tool API、权限矩阵	docs/specs/api_spec.md

2.2 Product Spec 摘要

用户故事	验收标准	实现/验证点
项目经理输入“分析当前高风险项目”	系统输出风险项目、分值和原因	analyze_project_risks + benchmark risk-analysis
管理者输入“生成本周经营周报”	系统输出指标、风险和待办建议	get_dashboard_metrics + generate_weekly_report
实施负责人输入“把 4 号紧急工单自动派单”	系统更新工单负责人并留痕	route_ticket + audit_logs
财务专员输入“通过合同 HT-2026-003”	系统审批合同并记录操作	approve_contract + approvals/contracts
评审者查看工具调用轨迹	能看到 run、step、tool_args、observation	agent_runs + agent_steps + 前端可观测日志

2.3 Architecture Spec 摘要

图 2-1 Architecture Spec 中的总体结构摘要

用户/评审

- > Web 前端
- > Python HTTP API
- > Session + RBAC
- > 企业业务模块
- > 智能运营 Agent
 - > 免费 DeepSeek 兼容 API Planner / 本地规则回退
 - > Function Calling 工具层
 - > 长期记忆 agent_memories
 - > 运行轨迹 agent_runs / agent_steps
- > SQLite 数据库与知识库

2.4 API Spec 摘要

接口类别	路径示例	说明
鉴权接口	POST /api/login, POST /api/logout, GET /api/me	Session Cookie + RBAC
业务接口	/api/customers, /api/projects, /api/contracts, /api/tickets	企业后台基础资源
Agent 接口	POST /api/agent/ask	输入自然语言任务, 返回 answer + steps
流式接口	GET /api/agent/stream	以 SSE 返回 plan/tool_start/tool_result/final
工具接口	GET /api/mcp/tools, POST /api/mcp/call	暴露 Function Calling 工具注册表
审计接口	GET /api/audit-logs	展示业务写操作和 Agent 工具调用留痕

2.5 需求到实现的追踪矩阵

需求编号	需求描述	实现文件	验证方式
R1	用户可登录并按角色访问系统	app/server.py, app/security.py	tests.py 登录 API
R2	Agent 可生成经营周报	app/agent/tools.py	benchmark weekly-report

R3	Agent 可分析项目风险	app/agent/tools.py	benchmark risk-analysis
R4	Agent 可执行写操作	route_ticket / approve_contract	smoke_web + 审计日志
R5	系统保存运行轨迹	app/agent/memory.py	agent_runs/agent_steps 查询
R6	MCP 工具服务可发现工具	scripts/mcp_server.py	tools/list JSON-RPC
R7	报告包含封面、目录、七章内容	docs/CS599_大作业报告_程辉高.docx	validate_report_docx.py

三、系统架构与设计

3.1 总体架构

系统采用“传统企业应用 + Agent 智能层”的组合架构。传统企业应用负责数据准确性、权限控制和业务流程；Agent 智能层负责理解用户目标、选择工具、组织多步骤推理并生成可读结果。这样的分层能避免 Agent 直接绕过业务系统，同时让传统后台拥有更自然的交互入口。

层级	模块	职责	关键文件
表现层	Web 前端	登录、看板、业务表格、智能助手、可观测日志	app/static/*
控制层	HTTP API	路由、请求解析、Session、RBAC、JSON 响应	app/server.py
业务层	客户/项目/合同/工单/审批	企业数据管理和流程动作	app/schema.sql
Agent 层	EnterpriseAgent	规划工具、执行工具、组织答案、写入记忆	app/agent/agent.py
工具层	Tool Registry	对业务能力进行 Function Calling 封装	app/agent/tools.py
记忆与追踪	Memory/Trace	保存对话记忆、run、step 和 observation	app/agent/memory.py
数据层	SQLite	保存业务、知识库、审计、Agent 运	app/database.py

		行数据	
--	--	-----	--

3.2 Agent 交互流程

步骤	动作	状态/数据	可观测证据
1	用户在智能助手输入自然语言任务	message	前端请求 /api/agent/ask
2	系统创建 Agent run 并读取最近记忆	run_id, recent_memory	agent_runs, agent_memories
3	Planner 生成工具计划	[(thought, tool_name, args)]	plan 事件
4	逐步调用工具注册表	tool_args -> observation	tool_start/tool_result 事件
5	记录每个工具步骤	thought, args, observation	agent_steps
6	组合最终答复并写入记忆	answer	assistant memory + run status

3.3 数据流设计

数据对象	来源	处理方式	输出/用途
经营指标	customers/projects/tickets/approvals	get_dashboard_metrics 聚合	经营看板和周报
项目风险	projects/contracts/tickets	analyze_project_risks 计算风险分值	风险预警和解释
工单分派	tickets/users	route_ticket 按角色和负载选择负责人	更新工单状态并审计
审批动作	contracts/approvals	approve_contract 检查合同号和决策	更新审批与合同状态
规则知识	knowledge_base	search_knowledge 关键词召回	Agentic RAG 答复增强
运行轨迹	Agent 执行过程	record_step 持久化	前端可观测日志与 benchmark 分析

3.4 数据库设计

表名	用途	关键字段
users	登录用户与角色	username, password_hash, role
customers	客户台账	name, industry, owner_id, status
projects	项目台账	customer_id, stage, risk_level, progress
contracts	合同与金额	contract_no, amount, status, project_id
tickets	问题/缺陷/任务工单	project_id, priority, status, assignee_id
approvals	审批流	entity_type, entity_id, status, approver_id
knowledge_base	企业规则知识	title, category, content
agent_memories	跨轮对话记忆	user_id, role, content
agent_runs	Agent 运行记录	message, status, answer
agent_steps	工具调用步骤	thought, tool_name, tool_args, observation
audit_logs	审计日志	actor_id, action, entity, detail

3.5 改造前 vs 改造后

维度	改造前	改造后	收益
交互方式	用户进入多个菜单手动筛选	用户提出自然语言目标	减少操作路径
风险分析	依赖人工跨项目、合同、工单判断	Agent 自动汇总并解释原因	提高风险识别一致性
工单派单	项目经理凭经验选择负责人	工具按规则和负载自动分派	降低延误和分配不均
合同审批	人工进入审批页面逐项处理	Agent 调用审批工具并审计	提升流程效率
报告生成	人工复制指标和待办	一键生成经营周报	缩短管理报告时间
可解释性	只记录部分操作日志	保存 run/step/tool observation	便于审计与答辩展示

四、关键实现与代码展示

4.1 Agent 核心循环

EnterpriseAgent 是系统的智能编排核心。它不直接在自然语言层面“猜答案”，而是先生成工具调用计划，再把每个工具结果作为 observation 记录下来，最后根据 observation 组合最终答复。这种结构接近 ReAct 的 thought/action/observation 模式，也方便后续替换为 LangGraph 状态图。

代码 4-1 Agent 核心循环摘要

```
memory = load_recent_memory(connection, user_id)
run_id = create_run(connection, user_id, message)
plan = deepseek_planner.plan(message, memory) or fallback_planner.plan(message, memory)

for index, (thought, tool_name, tool_args) in enumerate(plan, start=1):
    observation = call_tool(tool_name, tool_args, connection, user)
    record_step(connection, run_id, index, thought, tool_name, tool_args, observation)

answer = compose_answer(message, observations)
finish_run(connection, run_id, answer)
remember(connection, user_id, "assistant", answer)
```

4.2 Planner 设计：免费 DeepSeek API + 本地规则回退

系统优先读取 FREE_DEEPSEEK_API_KEY 或 SILICONFLOW_API_KEY，使用免费 DeepSeek 兼容 API 生成 Tool Calls。若未配置免费 Key，可读取官方 DEEPSEEK_API_KEY 作为备用；若远程接口不可用、额度不足、超时或返回非法工具，系统自动回退到 RuleBasedPlanner。这样既能展示真实 LLM Function Calling，又能保证答辩现场稳定。

配置项	用途	默认/说明
FREE_DEEPSEEK_API_KEY	免费 DeepSeek 兼容 API Key	推荐使用，不能提交到仓库
FREE_DEEPSEEK_BASE_URL	兼容 API 地址	https://api.siliconflow.cn/v1
FREE_DEEPSEEK_MODEL	远程模型名称	deepseek-ai/DeepSeek-V3.2
DEEPSEEK_API_KEY	官方 DeepSeek API Key	备用，官方接口按 token 计费
AGENT_DISABLE_LLM	禁用远程 LLM	设置为 1 后仅使用本地规则

4.3 RuleBasedPlanner 实现依据

RuleBasedPlanner 是面向演示场景的 deterministic fallback planner，基于关键词意图识别和参数抽取实现。它并不是最终智能能力的上限，而是为了在无 API Key、网络异常或余额不足时保证系统仍能完成核心闭环。其规则覆盖周报、风险、派单、工单查询、合同审批、客户/项目查询等课程演示重点。

意图关键词	工具计划	说明
周报/报告/经营	get_dashboard_metrics -> analyze_project_risks -> generate_weekly_report	先读指标，再识别风险，最后生成周报
风险/预警/延期/阻塞	search_knowledge -> analyze_project_risks	先检索规则，再计算风险
派单/分派/指派/路由	route_ticket 或 search_tickets	有工单号则写入派单，无工单号则先查询
合同/审批/通过/驳回	approve_contract 或 search_knowledge	识别合同号和审批决策
客户/项目	search_projects	按风险等级或关键词查询项目台账

4.4 工具定义与 Function Calling

工具名称	输入参数	业务动作	风险控制
get_dashboard_metrics	无	统计客户、项目、工单、审批、合同金额	只读
search_projects	keyword, stage, risk_level	查询项目台账	只读
search_tickets	status, priority, project_keyword	查询工单	只读
route_ticket	ticket_id	自动分派工单并更新状态	写操作，记录审计日志
analyze_project_risks	无	计算项目风险分值和原因	只读，输出解释
generate_weekly_r	无	生成经营周报	组合只读工具结果

eport			
approve_contract	contract_no, decision, comment	审批或驳回合同	写操作，记录审计日志
search_knowledge	query	检索企业规则知识	只读，限制在知识库

4.5 MCP 工具服务

除 Web API 外，项目还提供 scripts/mcp_server.py，以 stdio JSON-RPC 方式暴露 initialize、tools/list 和 tools/call 方法。外部 Agent 客户端可以通过该服务发现工具 schema 并调用业务工具，体现课程要求中的 MCP 协议融合。

代码 4-2 MCP stdio JSON-RPC 调用示例

```
{ "jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {} }
{ "jsonrpc": "2.0", "id": 2, "method": "tools/list", "params": {} }
{ "jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": { "name": "search_projects", "arguments": { "risk_level": "high" } } }
```

4.6 AI IDE 使用截图与 Demo 截图

图 4-1 AI IDE / 开发环境使用截图（已裁剪敏感 API Key 区域）

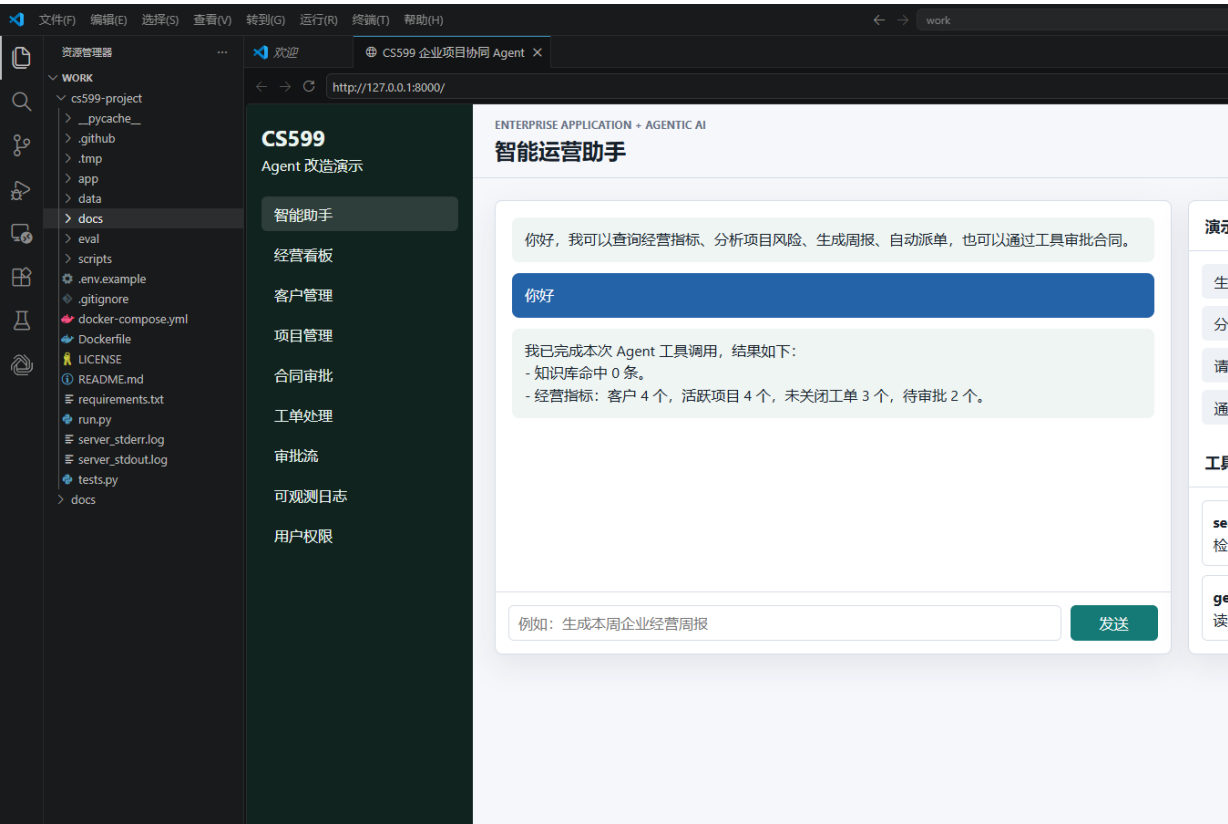
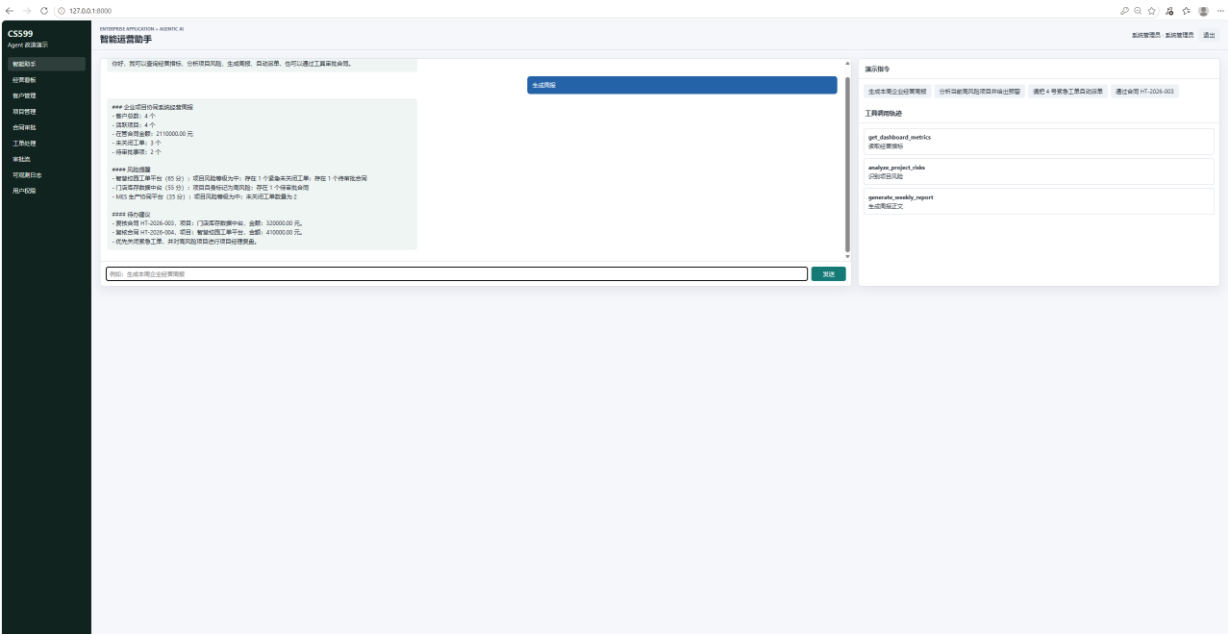


图 4-2 智能助手生成经营周报 Demo 截图



五、测试与评估

5.1 测试策略

测试设计遵循“业务功能 + Agent 行为 + Web 可用性 + 文档结构”的组合思路。传统企业应用需要验证登录、权限和 API；Agent 系统还要验证是否选择了正确工具、是否产生可检查的业务结果、是否保留运行轨迹。因此项目不只看最终回答是否流畅，还把工具调用命中率和 observation 作为评估对象。

测试类别	命令	验证内容	通过标准
语法检查	python -m compileall app scripts tests.py	检查 Python 文件可编译	无语法错误
功能测试	python tests.py	密码哈希、Agent 工具、登录 API、看板 API、Agent API	全部断言通过
Agent Benchmark	python scripts/benchmark_agent.py	风险分析、周报、派单三类任务	3/3 用例通过
Web Smoke	python scripts/smoke_web.py	临时启动服务并访问首页、登录、看板、Agent API	返回 status=ok
文档校验	python scripts/validate_report_docx.py	封面字段、标题、TOC 域、占位符	passed=True

5.2 Benchmark 设计

用例编号	Prompt	期望工具链	评估重点
risk-analysis	分析当前高风险项目并给出预警	search_knowledge, analyze_project_risks	是否先检索规则并识别风险
weekly-report	生成本周企业经营周报	get_dashboard_metrics, analyze_project_risks, generate_weekly_report	是否完成指标、风险、报告三步
ticket-routing	请把 4 号紧急工单自动派单	route_ticket	是否识别工单号并执行

			写操作
--	--	--	-----

指标	计算方式	意义
tool_score	期望工具在实际工具列表中的命中比例	衡量 Agent 是否调用了正确工具
keyword_score	最终答复中关键业务词的命中比例	衡量回答是否包含关键业务信息
passed	tool_score 与 keyword_score 是否达到阈值	给出可自动判断的行为评估结果
planner	记录使用 free-deepseek-api / deepseek-official-api / local-rule	便于解释远程 API 与回退行为

5.3 当前验证结果

验证项	结果	说明
compileall	通过	app、scripts、tests.py 均可编译
tests.py	通过	功能测试全部通过
benchmark_agent.py	3/3 通过	风险分析、经营周报、工单派单均命中期望工具
smoke_web.py	通过	可临时启动 Web 服务并完成 Agent API smoke
DOCX 结构校验	通过	封面字段、目录域、标题导航和占位符检查通过
PDF 大纲	通过	已验证生成 PDF 包含 /Outlines 导航大纲

代码 5-1 验证结果摘要

```
benchmark_agent.py:
  passed: 3
  total: 3
  risk-analysis: tool_score=1.0, keyword_score=1.0
  weekly-report: tool_score=1.0, keyword_score=1.0
  ticket-routing: tool_score=1.0, keyword_score=0.5

smoke_web.py:
  status: ok
  planner: local-rule
  tools: get_dashboard_metrics, analyze_project_risks, generate_weekly_report
```

5.4 Agent 行为案例分析

场景	Agent 行为	业务可验证结果
生成经营周报	调用指标、风险分析、周报生成三个工具	答复包含客户数、活跃项目、未关闭工单、待审批、风险提醒和待办建议
分析高风险项目	先检索风险规则，再分析项目风险	输出项目名称、风险分值和原因列表
自动派单	解析工单号并调用 route_ticket	更新 ticket.assignee_id/status 并写入 audit_logs
合同审批	解析合同号和决策并调用 approve_contract	更新合同/审批状态并写入审计

六、系统升级与扩展

6.1 架构演进路线

阶段	升级方向	预期收益	实施难点
短期	将 RuleBasedPlanner 的规则覆盖更多业务意图	提高离线演示和异常回退能力	规则维护成本增加
中期	引入 LangGraph 状态图	支持条件分支、失败重试、人工确认节点	需要重新抽象状态和边条件
中期	知识库接入向量数据库	从关键词 RAG 升级为语义 RAG	需要文档切分、嵌入模型和召回评估
长期	接入 OpenTelemetry / LangSmith 类 tracing	形成 token、延迟、工具质量的 LLMOps 面板	需要统一 trace schema
长期	部署到云服务器并接入统一认证	满足企业级运行与课程加分项	需要 HTTPS、配置管理和安全加固

6.2 生产化增强

- 安全增强：对写操作工具增加二次确认、权限判断和参数白名单，避免 Agent 越权执行。
- 可靠性增强：为远程 LLM 调用增加重试、超时、熔断和降级策略，并把失败原因写入 trace。

- 评估增强：扩展 benchmark 至更多任务类型，例如合同审批、知识库问答、复杂项目组合分析。
- 数据增强：把 SQLite 替换为 PostgreSQL/MySQL，并增加迁移脚本和数据备份策略。
- 体验增强：在前端显示 Planner 类型、每步工具耗时、输入参数和输出 observation，使评审能直观看到 Agentic AI 闭环。

6.3 AI 能力演进路径

能力层	当前实现	下一步演进
意图理解	规则 Planner + 免费 DeepSeek 兼容 API	多模型路由和领域 Prompt 模板
工具调用	Function Calling 工具注册表	工具权限、事务回滚和人工确认
记忆机制	agent_memories 保存近期上下文	用户画像、任务历史摘要和长期记忆检索
RAG	knowledge_base 关键词检索	向量检索、重排序、引用溯源
可观测性	agent_runs/agent_steps/audit_logs	LLMOps dashboard 和质量评分
多 Agent 协作	单 Agent 编排工具	项目经理 Agent、财务 Agent、运维 Agent 分工协作

七、课程总结

7.1 个人收获

通过本次项目，我对“企业级应用软件设计与开发”的理解从页面和接口层面进一步扩展到智能体编排层面。传统企业系统强调数据准确、流程稳定、权限清晰；Agentic AI 强调目标理解、工具调用、多步骤推理和结果解释。两者结合时，关键不是让大模型直接接管系统，而是把企业能力以安全、可审计、可评估的工具形式交给 Agent 编排。

SDD 方法也改变了我的开发节奏。以前容易先实现页面，再根据实现补文档；这次先写 Product Spec、Architecture Spec 和 API Spec，再用代码、测试、报告去验证规格，使项目边界更清楚，答辩时也能从需求、设计、实现、评估四个层次解释系统。

7.2 工程思维转变

转变前	转变后	体现
关注代码能否运行	关注规格、权限、审计、评估是否闭环	从单点功能走向工程交付
把 AI 当作问答接口	把 AI 当作工具编排器	使用 Function Calling 和 MCP
只看最终自然语言回答	检查工具调用轨迹和 observation	引入 benchmark 和 tracing 表
依赖在线 API	设计本地回退和可离线演示路径	RuleBasedPlanner 保证稳定性

•